CloudEco Wildfire Detection

FIT5225 Assignment 1 — Interview Preparation Guide

Model 1: Wildfire & Smoke Detection | FastAPI + Docker + Kubernetes + Terraform

Everything you need to explain your code confidently in the demo interview

Project File Map — Know Where Everything Is

File / Location	What it is	Open it in
app/main.py	FastAPI routes (/api/predict, /api/annotate)	VS Code / nano
app/predictor.py	YOLO model loader + inference logic	VS Code / nano
app/schemas.py	Pydantic request/response shapes	VS Code / nano
app/utils.py	Base64 ↔ OpenCV image conversion helpers	VS Code / nano
Dockerfile	Container build instructions	VS Code / nano
requirements_a1.txt	Python dependencies list	VS Code / nano
k8s/deployment.yaml	Kubernetes Deployment manifest	VS Code / nano
k8s/service.yaml	Kubernetes NodePort Service manifest	VS Code / nano
locust/locustfile.py	Load-test script for Locust	VS Code / nano
scripts/test_request.py	One-shot manual test script	VS Code / nano
teraform/*.tf	Terraform IaC for OCI VMs + networking	VS Code / nano

How to Open and Run the Project

1. Open VS Code → File → Open Folder → select the **wildfire-detection** folder on your Desktop.
2. In the VS Code terminal (Ctrl+`) or any PowerShell window navigate to that folder:

```
cd C:\Users\surya\OneDrive\Desktop\wildfire-detection
```

3. The sub-folders are: **app/** (Python service), **k8s/** (Kubernetes YAML), **locust/** (load test), **teraform/** (IaC), **fire-models/** (YOLO weights).

Section 1: app/main.py — FastAPI Entry Point

This is the **heart of the web service**. It creates the FastAPI application, loads the YOLO model once at startup, and defines the two API endpoints that clients call.

app/main.py — full file

```
from fastapi import FastAPI, HTTPException
from fastapi.concurrency import run_in_threadpool
from app.schemas import PredictionRequest, PredictResponse, AnnotateResponse
from app.utils import decode_base64_image, encode_image_to_base64
from app.predictor import YoloPredictor
MODEL_PATH = "fire-models/fire_m.pt"
app = FastAPI(title="CloudEco Wildfire Detection API", version="1.0.0")
# Load model ONCE at startup — not on every request
predictor = YoloPredictor(MODEL_PATH)
@app.get("/")
def root():
    return {"message": "Wildfire Detection API is running"}
@app.post("/api/predict", response_model=PredictResponse)
async def predict(request: PredictionRequest):
    try:
        image = decode_base64_image(request.image)   # base64 → OpenCV
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    try:
        # run_in_threadpool keeps YOLO off the async event loop
        result = await run_in_threadpool(predictor.predict, image)
        return predictor.build_predict_response(result, request.uuid)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Inference failed: {e}")
@app.post("/api/annotate", response_model=AnnotateResponse)
async def annotate(request: PredictionRequest):
    try:
        image = decode_base64_image(request.image)
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    try:
        result = await run_in_threadpool(predictor.predict, image)
        annotated = predictor.get_annotated_image(result)
        encoded   = encode_image_to_base64(annotated)
        return {"uuid": request.uuid, "annotated_image": encoded}
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Annotation failed: {e}")
```

Line-by-Line Explanation

from fastapi import FastAPI, HTTPException

→ Imports the core class to create the web server and HTTPException to send error responses with proper HTTP status codes.

from fastapi.concurrency import run_in_threadpool

→ THE KEY CONCURRENCY TRICK. FastAPI runs async event loop. YOLO prediction is CPU-bound and blocks. run_in_threadpool pushes it to a background thread so the event loop stays free for other requests.

MODEL_PATH = 'fire-models/fire_m.pt'

→ Points to the pre-trained wildfire YOLO weights file. This is the actual trained neural network that detects Fire and Smoke classes.

predictor = YoloPredictor(MODEL_PATH) [at module level]

→ The model is loaded ONCE when the server starts, not inside the route function. Loading YOLO takes 2-5 seconds. If we loaded it per-request, every call would be slow. This is a critical optimisation.

@app.get('/')

→ A simple health-check endpoint. Kubernetes readiness/liveness probes ping this URL to know if the pod is alive. Returns JSON confirming the API is running.

@app.post('/api/predict', response_model=PredictResponse)

→ Defines the main inference endpoint. response_model=PredictResponse means FastAPI will validate the output against the Pydantic schema — any missing field raises an error automatically.

async def predict(request: PredictionRequest)

→ The function is async, which means FastAPI can accept other requests while this one is waiting. The PredictionRequest type hint tells FastAPI to parse and validate the incoming JSON automatically.

image = decode_base64_image(request.image)

→ Converts the base64 string from the JSON body into an OpenCV NumPy array that YOLO can process.

result = await run_in_threadpool(predictor.predict, image)

→ Runs the YOLO inference in a thread pool, AWAITING completion without freezing the event loop. This is what separates a HD implementation from a C/D.

return predictor.build_predict_response(result, request.uuid)

→ Converts the raw Ultralytics result object into the exact JSON format required by the assignment spec.

Interview Questions for main.py

Q: Why did you use run_in_threadpool instead of just calling predictor.predict(image) directly?

YOLO inference is a CPU-bound operation — it cannot yield control to the event loop. If I called it directly inside an async function, it would block the entire FastAPI event loop, meaning no other HTTP request could be handled while YOLO is running. run_in_threadpool moves the computation to a background thread from Python's thread pool, so the event loop can keep accepting new requests in parallel.

Q: Why do you load the model outside the route function?

Loading a YOLO model reads several hundred MB from disk and initialises neural network weights — this takes 2–5 seconds. If I loaded it inside predict() or annotate(), every single request would pay that startup cost. By loading it once at module startup, every request reuses the already-loaded model in memory, giving sub-second inference latency.

Q: What does HTTPException status_code=400 mean?

400 is 'Bad Request'. We raise it when the client sends an invalid or un-decodable base64 string. The client made an error. 500 is 'Internal Server Error' — raised when our inference code itself crashes unexpectedly.

Q: What does response_model=PredictResponse do?

It tells FastAPI to validate the function's return value against the PredictResponse Pydantic schema. If any field is missing or has the wrong type, FastAPI raises an error before sending the response. It also auto-generates the API documentation at /docs.

Section 2: app/predictor.py — YOLO Model Logic

This module wraps the Ultralytics YOLO library. It loads the model, runs inference, converts the result into the required JSON format, and uses a **threading.Lock** to protect against simultaneous access from multiple concurrent requests.

app/predictor.py — full file

```
from ultralytics import YOLO
import threading
class YoloPredictor:
    def __init__(self, model_path: str):
        self.model = YOLO(model_path)        # load weights once
        self.lock  = threading.Lock()         # protect against concurrent access
    def predict(self, image):
        with self.lock:                       # only 1 inference at a time
            results = self.model.predict(
                source=image,
                device="cpu",
                verbose=False
            )
        return results[0]                     # Ultralytics returns a list
    def build_predict_response(self, result, request_uuid: str):
        detections_output = []
        boxes_output = []
        names = self.model.names              # {0: "fire", 1: "smoke"}
        if result.boxes is not None:
            for box in result.boxes:
                cls_id     = int(box.cls[0].item())
                class_name = names[cls_id]
                confidence = float(box.conf[0].item())
                x1,y1,x2,y2 = box.xyxy[0].tolist()   # corner format
                width  = x2 - x1
                height = y2 - y1
                detections_output.append(class_name)
                boxes_output.append({
                    "x": round(x1,2), "y": round(y1,2),
                    "width": round(width,2), "height": round(height,2),
                    "probability": round(confidence,4)
                })
        speed = result.speed if result.speed else {}
        return {
            "uuid": request_uuid,
            "count": len(detections_output),
            "detections": detections_output,
            "boxes": boxes_output,
            "speed_preprocess_ms":  round(float(speed.get("preprocess",0.0)),2),
            "speed_inference_ms":   round(float(speed.get("inference",0.0)),2),
            "speed_postprocess_ms": round(float(speed.get("postprocess",0.0)),2),
        }
    def get_annotated_image(self, result):
        return result.plot()    # Ultralytics draws boxes on the image
```

Line-by-Line Explanation

self.model = YOLO(model_path)

→ Loads the pre-trained wildfire detection model from the .pt file (PyTorch weights). This happens once in __init__. The YOLO class comes from the ultralytics library.

self.lock = threading.Lock()

→ A mutex (mutual exclusion lock). Because multiple HTTP requests arrive concurrently and run_in_threadpool gives each a thread, they could try to call self.model.predict() simultaneously. The lock ensures only ONE inference runs at a time, preventing memory corruption or race conditions.

with self.lock:

→ Acquires the lock before running inference. The 'with' statement automatically releases the lock when the indented block finishes, even if an exception is raised.

device='cpu'

→ Forces YOLO to use the CPU instead of GPU. The VMs in this assignment don't have GPUs. Explicitly specifying 'cpu' avoids YOLO trying to detect and fail on a CUDA device.

verbose=False

→ Suppresses Ultralytics printing progress bars and logs to stdout on every inference call. Without this, every API request would flood the container logs.

results[0]

→ YOLO's predict() returns a list of results because it supports batch inference (multiple images at once). We always pass one image, so we take index [0].

names = self.model.names

→ A dictionary like {0: 'fire', 1: 'smoke'}. YOLO's boxes store an integer class ID (0 or 1). We use this dict to convert the ID to a human-readable label.

box.xyxy[0].tolist()

→ Returns [x1, y1, x2, y2] — the top-left and bottom-right corners of the bounding box in pixel coordinates. We convert to (x, y, width, height) format because the spec requires it.

result.speed

→ A dict automatically populated by Ultralytics with millisecond timings for each inference stage: 'preprocess', 'inference', 'postprocess'. The assignment spec requires us to return these.

result.plot()

→ A built-in Ultralytics helper that draws all detected bounding boxes and class labels directly onto the image as a NumPy array. We use this for the /api/annotate endpoint.

Interview Questions for predictor.py

Q: Why do you use threading.Lock() if run_in_threadpool already handles concurrency?

run_in_threadpool means MULTIPLE requests run SIMULTANEOUSLY in separate threads. Each thread calls self.model.predict() at the same time. The Ultralytics/PyTorch model is not thread-safe — concurrent calls to .predict() can corrupt internal state. The Lock serialises access: even if 10 requests arrive together, each YOLO inference runs one at a time. This trades some throughput for correctness and stability.

Q: What are the two detection classes your model outputs?

Fire (class ID 0) and Smoke (class ID 1). These come from Model 1 — Wildfire & Smoke Detection, assigned to Student IDs ending in 0 or 5.

Q: Why does result.boxes contain xyxy format but your API returns x, y, width, height?

YOLO stores bounding boxes in xyxy format (top-left corner x1,y1 and bottom-right corner x2,y2) because that is efficient for intersection calculations during training. The assignment spec requires top-left corner + width +

height format because it is more intuitive for frontend rendering. We convert: width = x2 - x1, height = y2 - y1.

Q: What happens when no objects are detected?

result.boxes will still exist but be empty, so the for loop never executes. detections_output stays [], boxes_output stays [], and count returns 0. The spec says: 'When the count is 0, boxes should be an empty array.' — this is handled automatically.

Section 3: app/schemas.py — Request & Response Shapes

Defines the exact JSON structure for every request and response using **Pydantic BaseModel**. FastAPI automatically validates all incoming and outgoing data against these schemas.

app/schemas.py — full file

```
from pydantic import BaseModel, Field
from typing import List
class PredictionRequest(BaseModel):
    uuid:  str = Field(..., description="Unique request ID")
    image: str = Field(..., description="Base64 encoded image")
class BoxResponse(BaseModel):
    x:           float
    y:           float
    width:       float
    height:      float
    probability: float
class PredictResponse(BaseModel):
    uuid:                str
    count:               int
    detections:          List[str]
    boxes:               List[BoxResponse]
    speed_preprocess_ms:  float
    speed_inference_ms:   float
    speed_postprocess_ms: float
class AnnotateResponse(BaseModel):
    uuid:            str
    annotated_image: str
```

Interview Questions for schemas.py

Q: What does Pydantic do in this project?

Pydantic is a data validation library. When FastAPI receives a request, it uses PredictionRequest to automatically parse the JSON body and validate that 'uuid' and 'image' fields exist and are strings. If a field is missing or wrong type, FastAPI returns a 422 Unprocessable Entity error without our code ever running. For responses, PredictResponse validates our output before it is sent.

Q: Why is 'image' typed as str and not bytes?

JSON is a text protocol — it cannot carry raw binary bytes. The client encodes the image as a Base64 string (which is all ASCII text) and puts that string in the JSON. Our code then decodes the Base64 string back to bytes and into an image array.

Q: What does Field(...) mean?

The ... (Ellipsis) means the field is REQUIRED — Pydantic will raise a validation error if it is missing. The description is used by FastAPI to populate the automatic Swagger UI at /docs.

Section 4: app/utils.py — Image Conversion Helpers

Two utility functions that convert images between Base64 strings (used in JSON) and OpenCV NumPy arrays (used by YOLO).

app/utils.py — full file

```
import base64, cv2, numpy as np
def decode_base64_image(base64_str: str) -> np.ndarray:
    """Base64 string → OpenCV image array"""
    try:
        image_bytes = base64.b64decode(base64_str)    # text → raw bytes
        np_arr      = np.frombuffer(image_bytes, np.uint8)  # bytes → NumPy
        image       = cv2.imdecode(np_arr, cv2.IMREAD_COLOR)# NumPy → image
        if image is None:
            raise ValueError("Image decoding failed")
        return image
    except Exception as e:
        raise ValueError(f"Invalid base64 image: {e}")
def encode_image_to_base64(image: np.ndarray) -> str:
    """OpenCV image array → Base64 string"""
    success, buffer = cv2.imencode(".jpg", image)   # image → JPEG bytes
    if not success:
        raise ValueError("Image encoding failed")
    return base64.b64encode(buffer.tobytes()).decode("utf-8")
```

Step-by-Step: decode_base64_image

1. base64.b64decode(base64_str) — converts the Base64 ASCII text back into raw binary bytes (e.g. JPEG file bytes).
2. np.frombuffer(image_bytes, np.uint8) — wraps those bytes in a 1D NumPy array. This is required because cv2.imdecode expects a NumPy array, not raw Python bytes.
3. cv2.imdecode(np_arr, cv2.IMREAD_COLOR) — decodes the compressed image bytes (JPEG/PNG) into a 3D NumPy array (height × width × 3 colour channels). IMREAD_COLOR means BGR colour mode.
4. if image is None — cv2.imdecode returns None if the bytes are corrupted or not a valid image. We raise an explicit error in that case.

Step-by-Step: encode_image_to_base64

1. cv2.imencode('.jpg', image) — compresses the NumPy image array into JPEG format, returning (success_bool, buffer_array).
2. buffer.tobytes() — converts the NumPy buffer into standard Python bytes.
3. base64.b64encode(...).decode('utf-8') — converts binary bytes to Base64 ASCII text that can be safely embedded in JSON.

Interview Questions for utils.py

Q: Why do you need NumPy in the decode pipeline? Can't cv2 read bytes directly?

cv2.imdecode requires a NumPy array as input, not raw Python bytes. np.frombuffer is the standard bridge: it creates a 1D uint8 array backed by the same memory as the bytes object (no copy). This is efficient and is the documented pattern for in-memory image decoding with OpenCV.

Q: Why does cv2.IMREAD_COLOR matter?

YOLO was trained on 3-channel colour images (RGB/BGR). If we used IMREAD_GRAYSCALE the image would have 1 channel, causing a shape mismatch error inside the model. IMREAD_COLOR ensures we always get a (H, W, 3) array.

Section 5: Dockerfile — Container Build Instructions

The Dockerfile packages the FastAPI app, model weights, and all dependencies into a single, portable container image. This is what gets pushed to the registry and pulled by Kubernetes.

Dockerfile — full file

```
FROM python:3.11-slim
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgl1 \
    libglib2.0-0 \
    && rm -rf /var/lib/apt/lists/*
# Layer-cache optimisation: copy deps first, source code second
COPY requirements_a1.txt .
RUN pip install --no-cache-dir --upgrade pip && \
    pip install --no-cache-dir torch torchvision \
        --index-url https://download.pytorch.org/whl/cpu && \
    pip install --no-cache-dir -r requirements_a1.txt
COPY app        ./app
COPY fire-models ./fire-models
COPY demo-images ./demo-images
COPY scripts    ./scripts
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Line-by-Line Explanation

FROM python:3.11-slim

→ Base image. 'slim' is a minimal Debian variant — no GUI tools, no unnecessary packages, significantly smaller than the full python:3.11 image. This is important for keeping the container lean.

WORKDIR /app

→ Sets the working directory inside the container. All subsequent COPY, RUN, CMD instructions happen relative to /app. If /app doesn't exist, Docker creates it.

ENV PYTHONDONTWRITEBYTECODE=1

→ Tells Python not to write .pyc bytecode cache files inside the container. These files waste space in a container image and serve no benefit since the container is not persisted between runs.

ENV PYTHONUNBUFFERED=1

→ Disables Python's output buffering. Without this, print() and log statements might not appear in 'docker logs' until a buffer fills up. Setting this ensures real-time log visibility.

apt-get install libgl1 libglib2.0-0

→ OpenCV requires these system-level shared libraries to function. libgl1 is the OpenGL rendering library. libglib2.0-0 is a fundamental GLib runtime. Without them, 'import cv2' crashes. '--no-install-recommends' avoids pulling in dozens of optional packages.

rm -rf /var/lib/apt/lists/*

→ Removes the apt package index cache after installing. This can save 20-50 MB from the image. It is a standard Docker optimisation — these lists are only needed during the RUN step.

COPY requirements_a1.txt . [BEFORE source code]

→ LAYER CACHE TRICK: Docker rebuilds only layers that changed. If we copy requirements first and then run pip install, Docker caches the entire install layer. On the next build, if only source code changed (not requirements), Docker reuses the cached pip layer — saving minutes of install time.

--index-url https://download.pytorch.org/whl/cpu

→ Installs the CPU-only version of PyTorch. The default PyTorch from PyPI includes CUDA GPU support, adding ~3 GB. Since our VMs have no GPU, we use the CPU wheel — saving several gigabytes of image size.

--no-cache-dir

→ Tells pip not to save downloaded packages to its local cache inside the container. These cached files would only waste space in the final image since we're not running pip again.

COPY app ./app [AFTER pip install]

→ Source code is copied AFTER dependencies. This preserves the layer cache for the expensive pip install step whenever only application code changes.

EXPOSE 8000

→ Documents that the container listens on port 8000. This is informational only — it doesn't actually publish the port. The actual port mapping is done with -p 8000:8000 in docker run, or via the Kubernetes Service.

CMD [...]

→ The default command that runs when the container starts. Launches uvicorn (ASGI server) serving app.main:app (the FastAPI app object in app/main.py), listening on all network interfaces (0.0.0.0) on port 8000.

Interview Questions for Dockerfile

Q: Why did you choose python:3.11-slim instead of python:3.11?

The full python:3.11 image includes development tools, documentation, and many optional utilities we don't need at runtime. The slim variant strips these out, reducing the base image by ~400 MB. This makes registry pushes/pulls faster, reduces attack surface, and speeds up Kubernetes pod scheduling.

Q: Explain the layer caching optimisation you used.

Docker builds images layer by layer. Each instruction creates a new layer, and Docker caches layers that haven't changed. By copying requirements_a1.txt before the application source code, the expensive pip install layer is only re-run when dependencies change. If I only edited main.py, Docker reuses the pip cache — a build that would take 5+ minutes takes ~10 seconds.

Q: Why install PyTorch with --index-url https://download.pytorch.org/whl/cpu?

The default pip install torch downloads the CUDA-enabled build which is ~2.5 GB. Our Kubernetes nodes are CPU-only VMs with no GPU. The CPU-only wheel is ~300 MB — roughly 8x smaller. This drastically reduces image size, pull time, and storage costs.

Q: What does CMD vs RUN do?

RUN executes a command DURING the image build (e.g., installing packages). CMD defines the command to run WHEN the container STARTS. There can only be one effective CMD. If the container is run with a different command (e.g., bash), CMD is overridden.

Section 6: k8s/ — Kubernetes Deployment & Service

Two YAML manifest files tell Kubernetes *what* to run (Deployment) and *how to expose it* (Service).

k8s/deployment.yaml — full file

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: wildfire-api
  labels:
    app: wildfire-api
spec:
  replicas: 1         # scale to 2, 4, 8 for benchmarking
  selector:
    matchLabels:
      app: wildfire-api
  template:
    metadata:
      labels:
        app: wildfire-api
    spec:
      containers:
        - name: wildfire-api
          image: wildfire-api:latest
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
          resources:
            requests:
              cpu: "1"
              memory: "2Gi"
            limits:
              cpu: "1"
              memory: "2Gi"
          readinessProbe:
            httpGet:
              path: /
              port: 8000
            initialDelaySeconds: 20
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /
              port: 8000
            initialDelaySeconds: 30
            periodSeconds: 15
```

k8s/service.yaml — full file

```yaml
apiVersion: v1
kind: Service
metadata:
  name: wildfire-api-service
spec:
  type: NodePort
```

```
  selector:
    app: wildfire-api        # routes to pods with this label
  ports:
    - protocol: TCP
      port: 80               # Service internal port
      targetPort: 8000       # Pod container port
      nodePort: 30080        # External access port on every node
```

Key Fields Explained

replicas: 1

→ Number of pod instances. For benchmarking you scale this: kubectl scale deployment wildfire-api --replicas=4. Each replica is an independent YOLO inference worker.

matchLabels: app: wildfire-api

→ The selector must match the pod template labels. Kubernetes uses these labels to know which pods belong to this Deployment. If they don't match, the Deployment creates pods it can never manage.

image: wildfire-api:latest

→ The Docker image to run. Built locally and loaded into the cluster with 'docker build' then 'docker save | ssh | docker load' or pushed to a registry.

imagePullPolicy: IfNotPresent

→ Don't pull from registry if the image already exists locally. Saves bandwidth when the image is preloaded on each node.

resources.requests

→ The minimum CPU/RAM the scheduler guarantees this pod. Kubernetes uses this to decide WHICH NODE to place the pod on.

resources.limits

→ The MAXIMUM CPU/RAM the pod can use. If it tries to exceed this, the CPU is throttled or the pod is OOMKilled (memory). Setting requests = limits makes the pod Guaranteed QoS class.

cpu: '1'

→ Exactly 1 vCPU — required by the assignment spec. This hardware restriction is the foundation for the benchmarking experiments. Each pod gets exactly 1 CPU core, so you can measure how QPS scales linearly with pod count.

memory: '2Gi'

→ 2 Gibibytes RAM. YOLO model weights are ~50 MB but PyTorch + inference can peak at 1-2 GB including image buffers and intermediate tensors.

readinessProbe

→ Kubernetes pings GET / every 10 seconds after 20 seconds. The pod only receives traffic when this probe returns 200 OK. This prevents requests going to a pod still loading the YOLO model.

livenessProbe

→ Kubernetes pings GET / every 15 seconds after 30 seconds. If it fails, Kubernetes RESTARTS the container. This self-heals stuck or crashed pods automatically.

NodePort: 30080

→ Opens port 30080 on EVERY node in the cluster. External traffic to :30080 is forwarded to port 8000 inside the pods. NodePort range must be 30000-32767 by default.

type: NodePort

→ Exposes the service externally. Alternative types: LoadBalancer (creates cloud load balancer, better for production), ClusterIP (internal only, no external access).

Interview Questions for Kubernetes

Q: How do you scale the deployment during the benchmark?

kubectl scale deployment wildfire-api --replicas=4 — I change the replica count. Kubernetes scheduler places new pods across the worker nodes based on available resources. Once the pods pass their readiness probe, the Service automatically routes traffic to them.

Q: Why do readiness and liveness probes use the same endpoint '/' ?

The root endpoint GET / is a fast, lightweight health check — it just returns a JSON string with no inference. Readiness probe uses it to prevent traffic routing until the pod has fully started and the YOLO model is loaded. Liveness probe uses it to detect if the FastAPI server itself has locked up or crashed.

Q: What is the difference between requests and limits?

Requests are what the scheduler uses to PLACE the pod — it guarantees this much resource is available. Limits cap what the pod can CONSUME. If a pod exceeds CPU limit, it is throttled (slowed). If it exceeds memory limit, it is killed and restarted (OOMKilled). Setting them equal (Guaranteed class) gives predictable, consistent performance important for benchmarking.

Q: How does a request reach a pod from the internet?

Client → Node's public IP on port 30080 → NodePort Service → kube-proxy routes to an available pod on port 8000 → FastAPI in the container → YOLO inference → response back through the same path.

Section 7: locust/locustfile.py — Load Testing

The Locust script simulates multiple concurrent users sending requests to the API. It measures QPS (queries per second), latency, and error rates.

locust/locustfile.py — full file

```
import base64, os, uuid
from locust import HttpUser, task, between
# Load image once at script startup (not per-request)
def load_image_base64():
    image_path = os.path.abspath(os.path.join(
        os.path.dirname(__file__), "..", "demo-images", "image0.jpeg"
    ))
    with open(image_path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")
IMAGE_B64 = load_image_base64()     # shared by all users
class WildfireUser(HttpUser):
    wait_time = between(1, 2)        # 1-2 sec pause between requests
    @task(3)
    def predict(self):
        payload = {"uuid": str(uuid.uuid4()), "image": IMAGE_B64}
        self.client.post("/api/predict", json=payload, name="/api/predict")
    @task(1)
    def annotate(self):
        payload = {"uuid": str(uuid.uuid4()), "image": IMAGE_B64}
        self.client.post("/api/annotate", json=payload, name="/api/annotate")
```

Line-by-Line Explanation

IMAGE_B64 = load_image_base64() [module level]

→ The image is loaded and Base64-encoded ONCE when Locust starts, then shared across all simulated users. Encoding per-request would waste CPU time in the load generator itself, distorting results.

class WildfireUser(HttpUser)

→ Defines a simulated user type. Locust creates N instances of this class, each representing one concurrent user. Each user runs tasks in a loop.

wait_time = between(1, 2)

→ After each task, the user waits a random 1-2 seconds before the next. This simulates realistic think time between user actions, preventing artificially perfectly-spaced requests.

@task(3) and @task(1)

→ Task weights. In a given iteration, predict() is chosen 3 out of 4 times, annotate() 1 out of 4. This reflects that prediction (no image rendering) is the primary use case. Annotations are heavier — less frequent is realistic.

str(uuid.uuid4())

→ Generates a new unique ID for every single request. This lets the server (and client logs) correlate each async request to its response. The assignment spec requires this field.

self.client.post(..., json=payload)

→ Locust's built-in HTTP client. Using json= automatically sets Content-Type: application/json and serialises the dict. Locust tracks response time and status code for every call.

How to Run Locust

```
Run Locust (from project root):
cd C:\Users\surya\OneDrive\Desktop\wildfire-detection
locust -f locust/locustfile.py --host http://<NODE_IP>:30080
# Then open browser at http://localhost:8089
# Set number of users and spawn rate, then click Start Swarming
```

Interview Questions for Locust

Q: Why does IMAGE_B64 stay at module level rather than inside the task?

Base64 encoding a JPEG is CPU work. If each of 100 concurrent users re-encoded the image on every request, Locust itself would become the CPU bottleneck, not the server. By encoding once, we ensure Locust spends all its resources generating HTTP requests, and all measured latency comes from the server, not the client.

Q: What metrics does Locust report?

Requests/s (QPS), median response time (ms), 95th percentile response time (p95 ms), average response time, number of failures, and failure rate. For the benchmark report we look at how these change as we increase concurrent users.

Q: What is the 'breaking point' you were looking for?

The point at which response times start growing exponentially (e.g., jump from 500ms to 5000ms) or HTTP 500/503 errors begin occurring. This is where the server queue depth exceeds its capacity — matching queuing theory M/M/1 saturation. We record the last stable user count before this happens.

Section 8: teraform/ — Infrastructure as Code (OCI)

Terraform scripts automatically provision the Oracle Cloud Infrastructure (OCI) resources: VCN (Virtual Cloud Network), subnet, internet gateway, security rules, and 3 VMs.

teraform/provider.tf — OCI authentication

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    oci = { source = "oracle/oci", version = ">= 6.0.0" }
  }
}
provider "oci" {
  tenancy_ocid     = var.tenancy_ocid
  user_ocid        = var.user_ocid
  fingerprint      = var.fingerprint
  private_key_path = var.private_key_path
  region           = var.region   # ap-melbourne-1
}
```

teraform/network.tf — key resources

```
# 1. VCN (Virtual Private Network)
resource "oci_core_vcn" "fit5225_vcn" {
  cidr_blocks  = ["10.0.0.0/16"]   # 65536 private IPs
}
# 2. Internet Gateway
resource "oci_core_internet_gateway" "fit5225_igw" {
  vcn_id  = oci_core_vcn.fit5225_vcn.id
  enabled = true
}
# 3. Route Table: 0.0.0.0/0 → internet gateway
resource "oci_core_route_table" "fit5225_public_rt" { ... }
# 4. Security List: allows SSH(22) + K8s ports (6443,10250,30000-32767)
resource "oci_core_security_list" "fit5225_sec_list" { ... }
# 5. Public Subnet
resource "oci_core_subnet" "fit5225_public_subnet" {
  cidr_block  = "10.0.0.0/24"   # 256 IPs
  route_table_id  = oci_core_route_table.fit5225_public_rt.id
  security_list_ids = [oci_core_security_list.fit5225_sec_list.id]
}
```

teraform/compute.tf — 3 VM instances

```
data "oci_core_images" "ubuntu_images" {
  operating_system         = "Canonical Ubuntu"
  operating_system_version = "22.04"
  shape = var.instance_shape   # VM.Standard.E5.Flex
  sort_by = "TIMECREATED"      # pick latest image
}
resource "oci_core_instance" "master" {
  shape = var.instance_shape
  shape_config { ocpus=4, memory_in_gbs=8 }   # 4 vCPU, 8 GB RAM
  create_vnic_details { assign_public_ip=true }
  source_details {
    source_id = data.oci_core_images.ubuntu_images.images[0].id
  }
```

```
  metadata = { ssh_authorized_keys = local.ssh_public_key }
}
# worker1 and worker2 follow same pattern
```

Interview Questions for Terraform

Q: What ports did you open in the security list and why?

Port 22 (SSH) — for remote management from any IP. Port 6443 — Kubernetes API server (kubectl commands from master to workers). Port 10250 — kubelet API, workers report status to master on this port. Ports 30000-32767 — NodePort range, this is where our API is exposed on port 30080. Port 8472 UDP — Flannel/VXLAN overlay networking for pod-to-pod communication across nodes. All inter-node traffic also allowed from 10.0.0.0/24.

Q: Why use a data source for the Ubuntu image instead of hardcoding an OCID?

OCIDs (Oracle Cloud IDs) for OS images change when Oracle releases updates (new Ubuntu patches). A data source dynamically queries OCI for the LATEST Ubuntu 22.04 image matching the instance shape, so the Terraform config stays valid over time without manual OCID updates.

Q: What is terraform.tfvars for?

It stores actual values for variables (OCIDs, file paths, credentials) that are specific to your OCI account. The variables.tf file defines the variable names and types without values. This separation means the TF code is shareable while sensitive values stay in tfvars (which should be in .gitignore).

Q: How do you apply the Terraform config?

cd teraform → terraform init (downloads OCI provider plugin) → terraform plan (shows what will be created) → terraform apply (creates resources). terraform destroy tears everything down.

Section 9: start_wildfire.ps1 — Startup Script Walkthrough

Running **.\start_wildfire.ps1** from the project root starts the entire environment automatically. The script executes 7 sequential steps and exits only when the API is reachable.

```
Run the startup script (from project root in PowerShell)
cd C:\Users\surya\OneDrive\Desktop\wildfire-detection
.\start_wildfire.ps1
```

Step-by-Step Breakdown

Step 1 — Start Docker Desktop

Checks if a process named 'Docker Desktop' is already running using Get-Process. If not found, it launches Docker Desktop via Start-Process using the full .exe path. This prevents opening duplicate Docker windows if it's already open.

```
$dockerProcess = Get-Process -Name 'Docker Desktop' -ErrorAction SilentlyContinue
if (-not $dockerProcess) { Start-Process $dockerDesktopPath }
```

Step 2 — Wait for Docker engine

Polls 'docker version' in a loop up to 60 times (every 5 seconds = up to 5 minutes). 'docker version' only exits with code 0 when the Docker daemon is fully running. If Docker never becomes ready it exits with code 1. This prevents the rest of the script from running against a daemon that isn't up yet.

```
for ($i = 1; $i -le 60; $i++) {
    docker version *> $null
    if ($LASTEXITCODE -eq 0) { $dockerReady = $true; break }
    Start-Sleep -Seconds 5
}
```

Step 3 — Navigate to project folder

Calls Set-Location (cd) to move into the project root. All subsequent relative paths (.\k8s\deployment.yaml, .\venv\...) resolve from this directory.

```
Set-Location $projectPath
```

Step 4 — Activate Python virtual environment

Runs the venv activation script so that python, pip, and locust commands use the project's isolated packages instead of the system Python. If the venv folder is missing the script exits with an error rather than silently using the wrong Python.

```
$venvActivate = Join-Path $projectPath 'venv\Scripts\Activate.ps1'
& $venvActivate
```

Step 5 — Wait for Kubernetes cluster

Polls 'kubectl get nodes' up to 60 times (every 10 seconds = up to 10 minutes). kubectl returns exit code 0 only when it can reach the Kubernetes API server. For Docker Desktop's built-in K8s this means waiting for the control plane to boot. The script then prints all nodes so you can confirm they are Ready.

```
for ($i = 1; $i -le 60; $i++) {
    kubectl get nodes *> $null
    if ($LASTEXITCODE -eq 0) { $kubeReady = $true; break }
```

```
    Start-Sleep -Seconds 10
}
kubectl get nodes   # prints node table
```

Step 6 — Apply Kubernetes manifests + scale

Applies deployment.yaml (creates/updates the Deployment object) and service.yaml (creates/updates the NodePort Service). Uses 'kubectl apply' not 'create' so the command is idempotent — it works even if the resources already exist. Then explicitly scales to 1 replica and waits 15 seconds for the pod to start before checking status.

```
kubectl apply -f .\k8s\deployment.yaml
kubectl apply -f .\k8s\service.yaml
kubectl scale deployment wildfire-api --replicas=1
Start-Sleep -Seconds 15
```

Step 7 — Show status and open docs

Runs kubectl get pods and kubectl get svc to print a final status table. Then calls Start-Process to open the FastAPI Swagger UI in the default browser at port 30080 (the NodePort). After this the script exits — the API continues running in Kubernetes.

```
kubectl get pods
kubectl get svc
Start-Process 'http://127.0.0.1:30080/docs'
```

Interview Questions for start_wildfire.ps1

Q: Why does the script poll 'docker version' instead of just sleeping a fixed time?

A fixed sleep like Start-Sleep 30 is fragile — Docker Desktop might take 10 seconds on a fast machine or 90 seconds after a reboot. Polling with 'docker version' makes the script self-adapting: it proceeds the instant Docker is ready, no sooner and no later. The 5-second interval and 60-attempt ceiling prevent an infinite loop if Docker fails to start.

Q: Why use 'kubectl apply' instead of 'kubectl create'?

kubectl create fails if the resource already exists (throws an AlreadyExists error). kubectl apply is declarative — it creates the resource if absent and updates it if present. This makes the script idempotent: running start_wildfire.ps1 a second time updates the deployment to whatever is in the YAML without errors.

Q: What happens if the virtual environment is missing?

The script calls Test-Path on the venv activation script path. If it returns false, the script prints an error message and exits with code 1 immediately. It never reaches kubectl or Docker, preventing confusing errors from running the wrong Python version.

Q: Why open 127.0.0.1:30080 and not localhost:8000?

Port 8000 is the port inside the container. The Kubernetes NodePort Service maps external port 30080 → internal port 8000. Accessing 30080 goes through the full K8s networking stack (Service → kube-proxy → pod), which is what we want to test. 127.0.0.1 works here because Docker Desktop's K8s exposes NodePorts on the local machine's loopback address.

Section 9b: Resource Planning — Why 6 Effective Cores, Not 8

This section explains the resource arithmetic behind the cluster setup and why only **6 pods** can be reliably scheduled even though the 2 worker nodes have 8 vCPUs total.

VM Configuration

Node	Role	OCPUs	RAM	Schedulable for pods?
master-35415940	K8s control plane	4	8 GB	No — runs API server, etcd, scheduler
worker1-35415940	Worker	4	8 GB	Yes — minus ~1 CPU for OS/kubelet overhead
worker2-35415940	Worker	4	8 GB	Yes — minus ~1 CPU for OS/kubelet overhead

The Core Arithmetic

- **Total cluster OCPUs:** 3 nodes × 4 OCPUs = 12 OCPUs
- **Master node OCPUs:** 4 OCPUs consumed entirely by control plane components — kube-apiserver, etcd (the cluster database), kube-scheduler, kube-controller-manager. No application pods are scheduled here.
- **Worker node raw capacity:** 2 workers × 4 OCPUs = 8 OCPUs
- **System overhead per worker:** ~1 OCPU is permanently reserved by the Linux OS kernel, kubelet (the node agent), kube-proxy (network rules), and Flannel (the CNI overlay network daemon). These run as system daemons on every node.
- **Effective schedulable CPUs:** 2 workers × (4 – 1 overhead) = **6 vCPUs available for application pods**
- **Pod limit:** Each pod requests and is limited to exactly 1 vCPU (set in deployment.yaml). Kubernetes will not schedule a pod on a node where CPU requests would exceed available CPU. With 6 schedulable vCPUs, a maximum of 6 pods can be reliably scheduled without resource pressure.

Why Not Use More OCPUs Per VM?

The assignment specification explicitly requires VMs with **4 cores and 8 GB RAM** each. Using larger VMs would exceed the assignment constraints and invalidate the benchmarking comparison — results would not be reproducible by the marker. The 4-core constraint is also mathematically important: with each pod capped at 1 vCPU, it creates a clean linear relationship between pod count and theoretical throughput, making the performance graphs interpretable.

Impact on the Benchmark Experiment

Pod Count	vCPUs used	Headroom left	Expected behaviour
1 pod	1 / 6	5 free	Low load, plenty of headroom, low latency
2 pods	2 / 6	4 free	Still comfortable, near-linear throughput gain
4 pods	4 / 6	2 free	Good throughput, minor contention with system daemons
6 pods	6 / 6	0 free	Fully saturated — any new pod goes Pending

8 pods	8 / 6	OVER	2 pods stay Pending; cluster cannot schedule them without evicting system daemons

Interview Questions for Resource Planning

Q: Why can you only reliably run 6 pods if you have 8 worker vCPUs?

The 2 worker nodes each have 4 OCPUs, giving 8 raw worker vCPUs. However, approximately 1 vCPU per worker is permanently consumed by system-level processes: the kubelet node agent, kube-proxy for iptables network rules, the Flannel CNI daemon for overlay networking, and the Linux kernel itself. Kubernetes tracks this as 'allocatable' CPU which is less than 'capacity'. With 2 × (4 - 1) = 6 allocatable vCPUs and each pod requesting 1 vCPU, a maximum of 6 pods can be scheduled before the scheduler marks new pods as Pending.

Q: What happens when you try to schedule 8 pods?

Kubernetes schedules as many pods as the allocatable CPU allows — up to 6. The remaining 2 pods stay in Pending state. kubectl get pods will show them as Pending. kubectl describe pod shows a 'Insufficient cpu' event in the Events section. The 6 running pods still serve traffic normally; the 2 pending ones simply never start.

Q: Why set CPU requests equal to CPU limits (both = '1')?

When requests equal limits, Kubernetes assigns the pod QoS class 'Guaranteed'. The pod is never CPU-throttled below its limit and is the last to be evicted under node pressure. This gives predictable, consistent performance for benchmarking. If limits were higher than requests, the pod could burst and interfere with neighbours, making latency measurements unreliable.

Section 10: kubectl Commands — Demo Cheat Sheet

Commands you WILL run during the interview demo. Practise these until they are second nature.

`kubectl get nodes`

List all cluster nodes with status. Shows master + 2 workers. All should be 'Ready'.

`kubectl get pods`

List all running pods with status (Running/Pending/CrashLoopBackOff).

`kubectl get pods -o wide`

Like above but also shows which NODE each pod runs on + pod IP.

`kubectl get deployments`

Shows Deployment name, READY pods, and UP-TO-DATE count.

`kubectl get services`

Lists Services with TYPE (NodePort), CLUSTER-IP, and PORT mappings.

`kubectl describe pod`

Detailed info: events, resource limits, probe status, container logs.

`kubectl scale deployment wildfire-api --replicas=4`

Scale to 4 pods. Used during benchmarking.

`kubectl logs`

Stream the pod's stdout — see uvicorn request logs and any errors.

`kubectl logs -f`

Follow logs in real time (-f flag).

`kubectl apply -f k8s/`

Apply all YAML files in the k8s/ directory. Creates/updates resources.

`kubectl delete deployment wildfire-api`

Remove the Deployment (and its pods).

`kubectl top pods`

Live CPU and memory usage per pod (requires metrics-server installed).

`kubectl get events --sort-by='.lastTimestamp'`

Shows cluster events sorted by time — useful for debugging.

Demo Run Order

1. kubectl get nodes → show 3 nodes (master + 2 workers) all Ready
2. kubectl get pods → show pods Running
3. kubectl get services → show NodePort 30080
4. curl http://:30080/ → show {'message': 'Wildfire Detection API is running'}
5. python scripts/test_request.py → show a real inference response with detections
6. locust -f locust/locustfile.py --host http://:30080 → open browser, start swarming
7. kubectl scale deployment wildfire-api --replicas=4 → show scaling
8. kubectl get pods → show 4 pods Running

Section 11: How Did This Happen? From Which Code & Where

Every behaviour the interviewer sees has an exact origin in a specific file and line. This section maps each observable outcome back to the code that caused it.

Master Trace: Observable → File → Line → Why

What you observe	Which file	Which line / function	Why it works that way
API starts when container boots	Dockerfile	Last line: CMD ["uvicorn", "app.main:app", ...	Docker executes CMD on container start. uvicorn imports app/main.p
Model loads once, not per request	app/main.py	Line: predictor = YoloPredictor(MODEL_PATH) (module-level, outside any function)	Python executes module-level code when the file is imported. T
YOLO model file is fire_m.pt	app/main.py	Line: MODEL_PATH = "fire-models/fire_m.pt"	Hard-coded path to the wildfire model. The file is copied into the conta
GET / returns health message	app/main.py	def root(): return {"message": "Wildfire Detection API is running"}	The @app.get("/") decorator registers this function as the handler for
POST /api/predict returns JSON with detections	app/main.py	async def predict(request: PredictionRequest):	The @app.post decorator registers this as the handler. FastAPI parse
Request body has uuid + image fields	app/schemas.py	class PredictionRequest(BaseModel): uuid: str; image: str	Pydantic BaseModel defines the expected JSON shape. FastAPI feed
Response has count, detections, speed_*	app/schemas.py	class PredictResponse(BaseModel):	FastAPI validates the dict returned by predict() against this schema b
base64 string → OpenCV image	app/utils.py	def decode_base64_image(base64_str):	Called from main.py line: image = decode_base64_image(request.im
Annotated image → base64 string	app/utils.py	def encode_image_to_base64(image):	Called from main.py in annotate(). imencode compresses array to JP
YOLO does not block other requests	app/main.py	result = await run_in_threadpool(predictor.predict, image)	run_in_threadpool() moves the CPU-bound call into a background thr
Only one YOLO inference at a time	app/predictor.py	self.lock = threading.Lock() then with self.lock: in predict()	Without the lock, two threadpool threads would race on the model. Th
Detected class names are 'fire', 'smoke'	app/predictor.py	names = self.model.names → class_id → name	self.model.names is {0: 'fire', 1: 'smoke'} built into the .pt file. cls
Boxes are x,y,width,height not x1,y1,x2,y2	app/predictor.py	x1,y1,x2,y2 = box.xyxy[0].tolist(); width = x2-x1, height = y2-y1	YOLO xyxy = top-left + bottom-right corners. The spec requires top-left + size format
Speed timings appear in response	app/predictor.py	speed = result.speed → speed.get(preprocess), speed.get(inference), speed.get(postprocess)	Ultralytics automatically profiles each stage and stores milliseconds for ea
Annotated image has boxes drawn on it	app/predictor.py	def get_annotated_image(result): result.plot()	result.plot() is a built-in Ultralytics method that renders all bounding b
Container uses python:3.11-slim	Dockerfile	FROM python:3.11-slim	slim removes GUI tools and docs saving ~400 MB vs full image. Not a
PyTorch is CPU-only	Dockerfile	pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu	The CPU wheel is ~200 MB vs ~2.5 GB for the CUDA version. OCI V
pip install runs before COPY app/	Dockerfile	COPY requirements_a1.txt . then RUN pip install -r ... then COPY app/ ./app/	Docker layer caching. If only app/ changes, the pip install layer is
Pod only gets traffic after model loads	k8s/deployment.yaml	readinessProbe: httpGet path:/ initialDelaySeconds:20	Kubelet waits 20s then polls GET /. Pod receives traffic only after
Pod restarts if FastAPI hangs	k8s/deployment.yaml	livenessProbe: httpGet path:/ initialDelaySeconds:30 periodSeconds:15	If GET / fails 3 consecutive checks, Kubernetes kills and restarts the c
Each pod is capped at 1 vCPU	k8s/deployment.yaml	resources.limits.cpu: "1" and resources.requests.cpu: "1"	Requests = limits → QoS class Guaranteed. K8s never evicts this po
Traffic is routed to port 30080	k8s/service.yaml	type: NodePort nodePort: 30080 targetPort: 8000	Maps host port 30080 on every node. kube-proxy intercepts packets
Locust image is loaded once not per request	locust/locustfile.py	IMAGE_B64 = load_image_base64() (module level)	Python executes this when Locust imports the file. All 100+ virtual use
Locust sends 3x more predict than /health	locust/locustfile.py	@task(3) def predict() and @task(1) def health()	Locust picks tasks proportionally to their weights. 3+1=4 total weight,

Section 11b: Architecture & Flow Summary

Full Request Flow — Trace Every Hop

Step	What Happens	Code Location
1. Client	Reads JPEG, base64-encodes it, builds JSON payload with curl/locustfile.py or scripts/test_request.py	
2. HTTP POST	POST /api/predict with JSON body → NODE_IP:30080	HTTP layer
3. NodePort Service	kube-proxy forwards :30080 → Pod :8000 (round-robin across replicas)	k8s/service.yaml
4. FastAPI routes	async def predict() receives PredictionRequest object	app/main.py:29
5. base64 decode	decode_base64_image() converts string → NumPy BGR array	app/utils.py:8
6. Thread offload	run_in_threadpool(predictor.predict, image) — YOLO runs in thread	app/main.py:39
7. Lock acquire	threading.Lock ensures only 1 YOLO inference at a time	app/predictor.py:17
8. YOLO inference	model.predict(source=image, device='cpu') → result object	app/predictor.py:18
9. Result parsing	build_predict_response extracts boxes, labels, speed metrics	app/predictor.py:26
10. JSON response	FastAPI serialises PredictResponse → HTTP 200 JSON	app/main.py:41

Concurrency Architecture

FastAPI uses **asyncio** — a single-threaded event loop that handles many connections. The problem: YOLO is CPU-bound and would BLOCK the loop. Solution: **run_in_threadpool** hands YOLO to Python's ThreadPoolExecutor. The event loop awaits the future without blocking — it can service other requests while YOLO runs in parallel threads. The **threading.Lock** then serialises actual model calls to prevent race conditions.

Why This Matters for Performance (HD Criteria)

- Without run_in_threadpool: 10 concurrent users → all 10 wait in serial, latency = 10× single inference time.
- With run_in_threadpool: 10 users → FastAPI accepts all connections, YOLO runs in threads (serialised by lock), requests queue and process one at a time but the event loop stays free.
- With 4 pods (replicas): 4 parallel YOLO processes, each with their own model copy → 4× throughput.
- Horizontal scaling (more pods) is the primary way to increase QPS since each pod is CPU-limited to 1 vCPU.
- Little's Law: $L = \lambda \times W$. At saturation, if throughput λ plateaus but new requests keep arriving, queue length $L \to \infty$, response time $W \to \infty$.

Section 12: Quick-Reference: What Does Each File Do?

File	One-line purpose	Key class/function
app/main.py	FastAPI app, 2 endpoints	predict(), annotate()
app/predictor.py	YOLO wrapper + result shaping	YoloPredictor.predict()
app/schemas.py	JSON shape validation	PredictionRequest, PredictResponse
app/utils.py	Base64 ↔ OpenCV conversion	decode_base64_image(), encode_image_to_base64()
Dockerfile	Container build recipe	CMD: uvicorn
requirements_a1.txt	Python dependency list	fastapi, ultralytics, opencv-python-headless
k8s/deployment.yaml	Run N pod replicas	resources.limits cpu:'1'
k8s/service.yaml	Expose pods on NodePort 30080	type: NodePort
locust/locustfile.py	Load test 2 endpoints	WildfireUser, @task(3), @task(1)
scripts/test_request.py	Manual single-request test	requests.post()
teraform/provider.tf	OCI auth config	provider 'oci'
teraform/network.tf	VCN, subnet, IGW, security	oci_core_vcn, oci_core_subnet
teraform/compute.tf	3 VMs (master+2workers)	oci_core_instance × 3
teraform/variables.tf	Variable declarations	var.tenancy_ocid, var.region
teraform/outputs.tf	Print VM IPs after apply	master_public_ip, worker_public_ip

Top 5 Things to Memorise

1. run_in_threadpool — WHY: YOLO is CPU-bound and blocks the async event loop without it.
2. threading.Lock — WHY: multiple threads share one model object; concurrent .predict() calls cause race conditions.
3. CPU-only PyTorch — WHY: saves ~2 GB of image size; OCI VMs have no GPU.
4. COPY requirements before source code — WHY: Docker layer caching; avoids re-running pip on every code change.
5. readinessProbe — WHY: prevents Kubernetes from routing traffic to a pod still loading the YOLO model weights.

Common Interview Traps

Trap: "'I used async def so it's automatically concurrent'"

Correct answer: WRONG. async def only helps if the code inside yields control (uses await). Calling YOLO directly inside async blocks the loop.

Trap: "'The Lock prevents all concurrency'"

Correct answer: The Lock only serialises YOLO inference. FastAPI's event loop still accepts and queues new connections. Multiple requests are in-flight simultaneously — only the CPU-bound inference step is serialised.

Trap: "'NodePort is only accessible from inside the cluster'"

Correct answer: WRONG. NodePort is accessible from OUTSIDE via :30080. ClusterIP is internal-only.

Trap: "'I chose python:3.11-slim to save space'"

Correct answer: CORRECT, but be specific: slim removes development tools and optional libraries, saving ~400 MB vs the full image.

Section 13: Build, Deploy & Test — Step-by-Step

Build the Docker image

```
docker build -t wildfire-api:latest .
# Must run from project root where Dockerfile is
```

Test locally (no Kubernetes)

```
docker run -p 8000:8000 wildfire-api:latest
# Then: python scripts/test_request.py
```

Load image onto Kubernetes nodes

```
docker save wildfire-api:latest | ssh ubuntu@ docker load
docker save wildfire-api:latest | ssh ubuntu@ docker load
docker save wildfire-api:latest | ssh ubuntu@ docker load
```

Deploy to Kubernetes

```
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl get pods # wait for Running
```

Verify API is reachable

```
curl http://:30080/
python scripts/test_request.py
```

Scale for benchmarking

```
kubectl scale deployment wildfire-api --replicas=2
kubectl scale deployment wildfire-api --replicas=4
kubectl scale deployment wildfire-api --replicas=8
```

Run Locust

```
locust -f locust/locustfile.py --host http://:30080
# Open http://localhost:8089 in browser
```

Terraform (IaC)

```
cd teraform
terraform init
terraform plan
terraform apply
terraform destroy # to teardown
```

requirements_a1.txt — What Each Package Does

`fastapi==0.115.0`	The web framework — routing, request parsing, validation, OpenAPI docs.
`uvicorn[standard]==0.30.6`	ASGI server that runs the FastAPI app. [standard] adds websockets and HTTP/2 support.
`ultralytics==8.3.0`	The Ultralytics YOLO library — loads .pt model, runs inference, provides result.plot().
`opencv-python-headless==4.10.0.84`	OpenCV without GUI. 'headless' means no display/X11 dependency — essential for containers.
`numpy==1.26.4`	N-dimensional array library. OpenCV returns NumPy arrays; YOLO also operates on NumPy.

`pillow==10.4.0`	PIL — Ultralytics uses Pillow internally for image manipulation.
`python-multipart==0.0.9`	Required by FastAPI for form data parsing (not strictly needed here but standard dependency).
`locust==2.31.6`	Load testing framework — included in the same requirements for convenience.
`requests==2.32.3`	HTTP client library — used by scripts/test_request.py to send test requests.